

Document Number: p2912r0

Date: 2023-06-13

Target: SG1, LEWG

Reply to: gorn@microsoft.com

p2912r0: Concurrent queues and sender/receivers

This paper explores extending interface of Buffered Queue (<https://wg21.link/p1958r0>) with async APIs conforming to Sender/Receiver model according to <https://wg21.link/p2300>. It also makes stylistic API changes to be more consistent with existing library facilities.

Additionally, we report on implementation experience (<https://github.com/GorNishanov/conqueue>) addressing the concern that supporting both synchronous and asynchronous push/pop in the same queue is a challenge (<https://wg21.link/p2882r0>). The answer based on the implementation is that it is no challenge at all.

Changes to `buffer_queue`

By following an example of `std::future`, enum `queue_op_status` was renamed to `conqueue_errc` and we introduced an exception `conqueue_error` that will be thrown to carry the `conqueue_errc`.

```
enum class conqueue_errc { success = 0, empty, full, closed };  
class conqueue_error : system_error { ... };  
... make_error_code, make_error_condition, conqueue_category, ...
```

The member functions `wait_pop` and `wait_push` were used as non-throwing versions of a blocking pop and push. By analogy with how `<filesystem>` deals with this case, we restyled them as follows:

```
T pop(); // used to be pop_value  
std::optional<T> pop(std::error_code &ec); // used to be wait_pop  
std::optional<T> try_pop(std::error_code &ec);
```

```

void push(const T& x);
bool push(const T& x, error_code &ec); // used to be wait_push
bool try_push(const T& x, error_code &ec);

void push(T&& x);
bool push(T&& x, error_code &ec); // used to be wait_push
bool try_push(T&& x, error_code &ec);

```

Finally, to support async push and pop, we added

```

sender auto async_push(const T& x) noexcept(is_nothrow_copy_constructible_v<T>);
sender auto async_push(T&& x) noexcept(is_nothrow_move_constructible_v<T>);
sender auto async_pop() noexcept;

```

Based on usage experience, we can consider adding asynchronous equivalents of other flavors of push and pop as needed.

Implementation experience

A demonstration implementation is available in <https://github.com/GorNishanov/conqueue>.

An implementation only requires some kind of critical section to be able to change several related values atomically. For example, a spinlock is sufficient.

Additionally, to implement blocking for synchronous push and pop, it is sufficient to use C++20's `std::atomic_flag` wait facilities.

The highlights of one possible implementation are:

```

class buffer_queue {
    ...
private:
    std::mutex mutex; // or spinlock of some sort
    detail::ring_buffer buffer;
    detail::intrusive_list<&pop_waiter::prev, &pop_waiter::next> pop_waiter_list;
    detail::intrusive_list<&push_waiter::prev, &push_waiter::next> push_waiter_list;
    bool closed{};
};

```

Common base for both synchronous and asynchronous waiters stores:

```
struct waiter_base {
    waiter_base* next{};
    waiter_base* next{};
    // For push case, we store the pointer to the value to be pushed.
    // For pop case, we store the value popped from the queue.
    variant<monostate, conqueue_errc, T, const T*, T*> value;
    void (*complete)(waiter_base*) noexcept;
};
```

And a concrete implementation for a synchronous waiter:

```
template <typename T> struct buffer_queue<T>::blocking_waiter : waiter
    std::atomic_flag flag;

    blocking_waiter() noexcept {
        this->complete = [](waiter_base* w) noexcept {
            auto *self = static_cast<blocking_waiter*>(w);
            // Notifying sync waiter.
            self->flag.test_and_set();
            self->flag.notify_one();
        };
    }

    blocking_waiter(T& x) noexcept : blocking_waiter() {
        this->value = std::addressof(x);
    }

    void wait() noexcept { flag.wait(false); }
};
```

Sender/receiver implementation is a bit more involved (as usual with sender/receivers unrelated to the task at hand), so here we are only showing the completion routine:

```
this->complete = [](waiter_base* w) noexcept {
    auto* self = static_cast<operation*>(w);
    if (auto* errc = get_if<conqueue_errc>(&self->value))
        stdexec::set_error((Receiver&&)self->receiver,
                           make_exception_ptr(conqueue_error(*errc)));
    else
```

```
stdexec::set_value((Receiver&&)self->receiver);  
};
```

Processing of the waiters is performed uniformly by interacting with `waiter_base` by setting or reading error or value from the variant and invoking `complete` to resume the sender or synchronous waiter. Having the queue support both synchronous and asynchronous APIs does not present a challenge.

T pop() vs void pop(T&)

In the original paper [buffer_queue paper](#), the `pop` function had signature `T pop_value()`. Subsequently, it was changed to `void pop(T&)` due to concern about the problem of losing elements.

Unlike STL's combinations of `void pop()` and `T& front()` that are possible for synchronous cases, such a solution does not work for concurrent queues, where we cannot observe the value before popping it from the queue.

Comparing `T pop()` and `void pop(T&)` we believe that they are equivalent from exception safety standpoint and `T pop()` wins on ergonomics of usage.

Naming wise, we chose `T pop()` rather than `T pop_value()` for consistency with the rest of the APIs and due to `[[nodiscard]]` guarding against misuse (thus if a user imagined that `pop` API is `void pop()` by analogy with `std::stack`, for example, a compiler error will quickly bring them to their senses).

try_push(T&&) vs try_push(T&&, T&)

In the original buffer queue paper [\[p1958r0\]](#), the `try_push` was:

```
queue_op_status try_push(Value&& x);
```

in the later paper [\[p0260r5\]](#), it was changed to:

```
queue_op_status try_push(Value&& x, Value& x);
```

with the rule:

If the queue is full or closed, return the respective status and move the element into the second parameter. Otherwise, push the element onto the queue and return `queue_op_status::success`.

The rationale is likely was to have an ability not to lose a temporary value if push operation did not succeed.

It seems that it is possible in both versions:

```
T x = get_something();  
if (try_push(std::move(x))) ...
```

With two parameter version:

```
T x;  
if (try_push(get_something(), x)) ...
```

Ergonomically they are roughly identical. API is slightly simpler with one argument version, therefore, we reverted to original one argument version.

Varna update

This paper was seen by SG1 in Varna and recommendation was to merge proposed changes into the next revision of <https://wg21.link/p0260>.

References

1. [p1958r0: C++ Concurrent Buffer Queue](#)
2. [p0260r5: A proposal to add a concurrent queue to the standard library](#)
3. [p2882r0: An Event Model for C++ Executors](#)
4. [p2300r7: std::execution](#)